

```
# =====  
# CONCEPT MAP: IMAGE SENSING MECHANISMS  
# Google Colab Code  
# =====  
  
# Install Graphviz  
!apt-get -qq install graphviz  
!pip -q install graphviz  
  
from graphviz import Digraph  
from IPython.display import Image, display  
  
# -----  
# Create Concept Map  
# -----  
dot = Digraph("ImageSensingMechanisms", format="png")  
  
# Layout Settings  
dot.attr(rankdir="TB")  
dot.attr(size="16,16")  
dot.attr(dpi="300")  
dot.attr(nodesep="0.5")  
dot.attr(ranksep="0.8")  
dot.attr(splines="ortho")  
  
# =====  
# MAIN NODE  
# =====
```

```
dot.node(  
    "A",  
    "IMAGE SENSING\nMECHANISMS",  
    shape="box",  
    style="filled",  
    fillcolor="lightblue",  
    fontsize="22"  
)  
  
# =====  
# IMAGE SENSING COMPONENTS  
# =====  
  
dot.node("B",  
    "Optical Lens",  
    shape="box",  
    style="filled",  
    fillcolor="lightgreen")  
  
dot.node("C",  
    "CCD / CMOS Sensor",  
    shape="box",  
    style="filled",  
    fillcolor="lightgreen")  
  
dot.node("D",  
    "Pixel Array",  
    shape="box",  
    style="filled",  
    fillcolor="lightgreen")
```

```
dot.node("E",
        "Analog-to-Digital\nConverter (ADC)",
        shape="box",
        style="filled",
        fillcolor="lightgreen")

# =====
# SENSOR CHARACTERISTICS
# =====

dot.node("F",
        "Sensor Characteristics",
        shape="box",
        style="filled",
        fillcolor="khaki")

dot.node("G","Pixel Size",shape="ellipse")
dot.node("H","Quantum Efficiency",shape="ellipse")
dot.node("I","Dynamic Range",shape="ellipse")
dot.node("J","Sensor Sensitivity (ISO)",shape="ellipse")

# =====
# NOISE SOURCES
# =====

dot.node("K",
        "Noise Generation",
        shape="box",
        style="filled",
        fillcolor="lightcoral")
```

```

dot.node("L","Thermal Noise",shape="ellipse")
dot.node("M","Shot Noise",shape="ellipse")
dot.node("N","Read Noise",shape="ellipse")
dot.node("O","Dark Current Noise",shape="ellipse")

# =====
# IMAGE ACQUISITION QUALITY
# =====

dot.node("P",
        "Image Acquisition Quality",
        shape="box",
        style="filled",
        fillcolor="lightskyblue")

dot.node("Q","High Signal-to-Noise Ratio",shape="ellipse")
dot.node("R","High Resolution",shape="ellipse")
dot.node("S","Accurate Color Reproduction",shape="ellipse")
dot.node("T","Improved Image Quality",shape="ellipse")

# =====
# APPLICATIONS
# =====

dot.node("U",
        "Computer Vision Applications",
        shape="box",
        style="filled",
        fillcolor="palegreen")

dot.node("V","Medical Imaging",shape="box")
dot.node("W","Autonomous Vehicles",shape="box")

```

```

dot.node("W", "Autonomous Vehicles", shape="box")
dot.node("X", "Remote Sensing", shape="box")
dot.node("Y", "Face Recognition", shape="box")
dot.node("Z", "Industrial Inspection", shape="box")

# =====
# LITERATURE
# =====

dot.node(
    "AA",
    "Literature Perspectives\n\n"
    "• Gonzalez & Woods\nDigital Image Processing\n\n"
    "• Richard Szeliski\nComputer Vision\n\n"
    "• Forsyth & Ponce\nComputer Vision\n\n"
    "• Anil K. Jain\nFundamentals of Digital Image Processing",
    shape="note",
    style="filled",
    fillcolor="lightyellow"
)

# =====
# CONNECTIONS
# =====

# Main workflow
dot.edge("A", "B", label="focuses light")
dot.edge("B", "C", label="onto")
dot.edge("C", "D", label="contains")
dot.edge("D", "E", label="converted by")

# Sensor characteristics

```

```
dot.edge("C","F",label="defined by")
dot.edge("F","G")
dot.edge("F","H")
dot.edge("F","I")
dot.edge("F","J")

# Characteristics → Quality
dot.edge("G","P",label="improves")
dot.edge("H","Q",label="increases")
dot.edge("I","R",label="supports")
dot.edge("J","S",label="affects")

# Noise
dot.edge("C","K",label="may generate")
dot.edge("K","L")
dot.edge("K","M")
dot.edge("K","N")
dot.edge("K","O")

# Noise impact
dot.edge("L","P",label="reduces")
dot.edge("M","P",label="reduces")
dot.edge("N","P",label="reduces")
dot.edge("O","P",label="reduces")

# Acquisition Quality
dot.edge("P","Q")
dot.edge("P","R")
dot.edge("P","S")
dot.edge("Q","T")
dot.edge("R","T")
```

```
dot.edge("S","T")

# Applications
dot.edge("T","U",label="supports")
dot.edge("U","V")
dot.edge("U","W")
dot.edge("U","X")
dot.edge("U","Y")
dot.edge("U","Z")

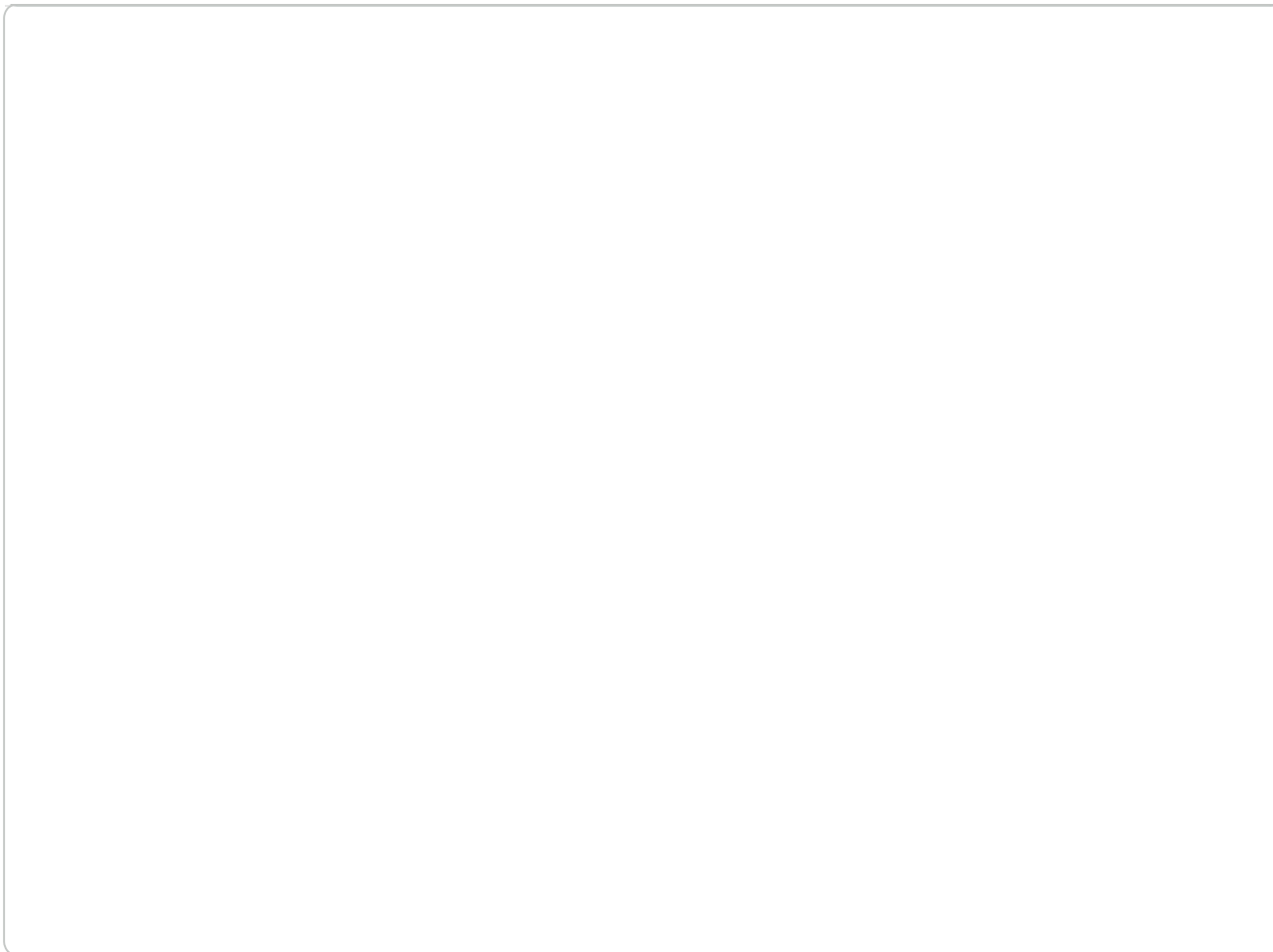
# Literature
dot.edge("AA","A",label="supported by")

# =====
# SAVE & DISPLAY
# =====

filename = dot.render("Image_Sensing_Mechanisms_Concept_Map")

display(Image(filename))

print("Concept Map generated successfully!")
```



Warning: Orthogonal edges do not currently handle edge labels. Try using xlabel.

